15-618 - Project Proposal

Anthony Tsiantis
atsianti@cs.cmu.edu

November 11, 2026

Contents

1 Title **2**

2 URL **2**

3 Summary **2**

4 Background **2**
4.1 Prefill vs Decode Attention 2
4.2 The Decode Attention Computation 2
4.3 What benefits from parallelism? 3

5 The Challenge **3**
5.1 Dependencies 4
5.2 Memory Access Characteristics 4
5.3 Divergence and Load Imbalance 4
5.4 System Constraints 4
5.5 What I hope to learn 4

6 Resources **5**
6.1 Hardware 5
6.2 Software and Starting Code 5
6.3 References 5

7 Goals and Deliverables **6**
7.1 60% Goal (If work goes slowly) 6
7.2 80% Goal (Plan to Achieve) 6
7.3 100% Goal (Plan to Achieve) 6
7.4 Stretch Goals (Hope to Achieve) 6
7.5 Analysis Questions 6

8 Platform Choice **7**

9 Schedule **7**

1 Title

The title of this project is "Parallelizing grouped-query decode attention over the KV cache". I, Anthony Tsiantis, will be doing this solo.

2 URL

The project website is available at `https://anthonytsiantis.github.io/decode-attention-618-website/`.

3 Summary

In this project, I will be building decode attention kernels in CUDA C++. I will first start with a naive baseline, then move to a optimizing a grouped query attention (GQA) kernel, then split the context across thread blocks with a combine step, and finally a paged KV cache. I will be using NVIDIA's 2080 GPUs and TinyLlama to test my kernels and measure speedup, improvements in arithmetic intensity and throughput when serving multiple requests.

4 Background

4.1 Prefill vs Decode Attention

Modern LLM inference has two steps: prefill and decode. The first step, prefill, is when all the tokens of a prompt are run through the model, producing their respective Q, K, V values, while running the attention mechanism. During this step, every token's K and V values are cached in the KV cache. Since each token's Q, K, V values are independent of other token's, this computation can be done concurrently within a layer. Since many tokens are processed together, each weight loaded from memory is reused across many tokens (high arithmetic intensity) and thus this is a compute bound problem.

Once prefill completes, we have the first output token and the decode step can start. The decode step generates one token at a time, and each token is dependent on the previous ones output. The new token's query attends to all cached K and V values. Therefore, each token needs to read the KV values of prior tokens. With only one query, each key and value loaded from memory is used for very little computation. Thus, this is a memory bound computation. For this project, I will focus on optimizing decode attention.

4.2 The Decode Attention Computation

In this section, I will briefly cover how the decode attention computation occurs, specifically with the TinyLlama model (at least how I have come to understand it). The first step, takes the token's hidden vector at this layer, x_t and multiplies

it by each of the weight matrices W_Q, W_K, W_V, to produce the query, key and value respectively. TinyLlama also applies positional encoding for the query and key in this step.

$$\mathbf{q}^{(h)} = \mathbf{x}_t W_Q^{(h)}, \qquad \mathbf{k}_t^{(g)} = \mathbf{x}_t W_K^{(g)}, \qquad \mathbf{v}_t^{(g)} = \mathbf{x}_t W_V^{(g)}$$

Here t is the position of the new token, q is the query (for one query head), k is the key, and v is the value.

In the second step, we identify which KV head each query head uses. With Grouped Query Attention (GQA), TinyLlama has 32 query heads sharing 4 KV heads, so 8 query heads per KV head. Following this, the previously computed new key and value are written to position t of the cache for KV head g. We can then use these values in the attention equation to compute the attention output for that head (which then gets combined with all the other heads' outputs).

$$\mathbf{o} = \text{softmax}\left(\frac{\mathbf{q}K^\top}{\sqrt{d}}\right)V$$

As an aside, 3Blue1Brown's attention and transformer YouTube videos were super helpful to understanding these mechanisms visually.

4.3 What benefits from parallelism?

This project has many opportunities for parallelism. My naive implementation will parallelize across heads. Since each query head's attention is independent, they can be run in parallel. Another benefit of parallelism is the GQA reuse. Since the 8 query heads sharing a KV head read the same keys and values, it can be loaded into shared memory once and all those computations can be performed in parallel.

As you can see the softmax equation in the previous section, there is an opportunity for parallelism here by splitting the context into chunks. With each token attending to all previous tokens, we can partition this work so that each thread block can work on a subset (chunk) of the workload at a time. While softmax does keep track of the global max and sum, partitions will keep track of their own local max and sum and the final combination step will handle merging the results. The combination step will combine the partial attention results from different chunks of the KV cache and merge them back into a single answer.

Finally, during production inference, it is expected that multiple users will be prompting a model at the same time. Each users request is independent of the other users and thus parallelism can be exploited among users.

5 The Challenge

Decode attention has very little parallelism per request and very little computation, so the challenge with this project is to keep the GPU's cores busy while maximizing arithmetic intensity by not wasting memory bandwidth.

5.1 Dependencies

Within decode attention there are several dependencies. Firstly, the softmax function needs the max and sum over all cached tokens, so when partitioning these computations into thread block sized chunks, we need to keep a local max and sum and have the final combine function handle the global max and sum. This final combine step becomes a synchronization point that must occur after all the computations have completed. Also, since an output at time t is a result of all outputs before it, tokens must be processed in sequential order and they must process through the model in sequential order because you cannot reorganize the model. These dependencies severely limit parallelism.

5.2 Memory Access Characteristics

The KV cache can get very large and thus is stored in GPU memory. Each decode step reads every cached key and value for the layer from GPU main memory once. With TinyLlama, there is approximately 22KB per token. As the context window grows, this exceeds the GPU cache (2K tokens is approx. 44MB across all layers) and since there is no reuse across steps, data is read from memory every time. Since we are not doing much work with the data we are reading in, I expect this to be memory bound (high communication to computation ratio). There are some ways to reduce the number of reads. To achieve this, it will be important to ensure that a warp reads consecutive addresses to make each memory operation efficient. Furthermore, since 8 query heads are sharing one KV head, one load can be reused 8 times, increasing computation per read.

5.3 Divergence and Load Imbalance

After partitioning the attention mechanism into chunks, the last chunk will most likely be a partial chunk so you will have some threads sitting idle. This is not an ideal load balance.

5.4 System Constraints

The properties of the 2080 may make mapping hard. For example, the 2080 has 46 SMs and TinyLlama has 32 query heads sharing 4 KV heads, 8 query heads per KV head. This implies that some SMs will be sitting idle. Furthermore, there is limited shared memory and registers per SM, which limits how much each block can hold and how many blocks can run at once. Another issue is that kernel launch overhead and the synchronization for the combine step may significantly limit speedup.

5.5 What I hope to learn

There are many things I would like to learn from this project. Some higher level ideas are actually deploying models and how inference works. This project will

also enable me to understand the nitty gritty of decode attention. Beyond this, I hope to answer my analysis questions below.

6 Resources

6.1 Hardware

I plan to run this code on the GHC machines as they have 2080s. One concern I have is exceeding my disk allowance on the GHC machines (if you know how to extend that please let me know!). If necessary, I will work on this project locally as I have a 2080 Super on my PC. For the stretch goal, I will need access to PSC GPUs (would likely need help to gain access to these).

6.2 Software and Starting Code

I plan to implement all the CUDA attention kernels, the combine step and the paged KV cache and block allocator, and the benchmarking code. I will build upon PyTorch for tensors, the CUDA extension system, an attention reference and use PyTorch's scaled dot product attention (SDPA) as a baseline. I will also use Hugging Face Transformers to load and run TinyLlama and TinyLlama-1.1B weights.

6.3 References

I will be reading several papers on the subject material to help me better understand how these mechanisms work as I am fairly new to inference. Some recommendations the LLMs have made me on the subject are:

Flash-Decoding
FlashInfer
vLLM PagedAttention
If you have any more I would love to hear your feedback!

7 Goals and Deliverables

7.1 60% Goal (If work goes slowly)

By the milestone deadline, I plan to have implemented a naive kernel that uses one thread block per query head, a GQA kernel that takes advantage of GPU shared memory and a fixed split kernel that chunks the computations with a synchronous combine step. Also, I plan to have these all verified against the PyTorch reference and bench marked across context lengths. If work goes slowly, I hope to at least have all these kernels implemented. With these kernels, the most important target is that they are correct! To validate correctness, I will be comparing the output against the PyTorch reference (within FP16 tolerance). Another target is that GQA kernels should read around 8x fewer KV bytes than the naive version since one KV read serves 8 query heads in shared memory.

7.2 80% Goal (Plan to Achieve)

I foresee optimal chunk size selection being dependent on context length. For this reason, one of my 80% goals is to automatically select an optimal chunk selection based on context length. Another goal is to implement batched decodes with requests of different lengths with good load balancing among the GPU's SMs. For these improvements, I am hoping to have my best kernel reach $\geq 50\%$ of 448 GB/s (memory bandwidth of the 2080) for long contexts. Furthermore, I hope that these kernels will have around a 4x speedup over my naive implementation on long contexts.

7.3 100% Goal (Plan to Achieve)

For my 100% goal, I hope to implement a paged KV cache integrated into TinyLlama. The goal with paged KV cache is to allocate memory on demand instead of all at once. Therefore, I hope to see an improvement in concurrent requests without sacrificing too much performance.

7.4 Stretch Goals (Hope to Achieve)

As a stretch goal, and if I can get access to the PSC GPUs, I may see if I can run my kernels on the PSC cluster's more advanced GPUs and see if a best split changes with different SM count and higher bandwidth of more advanced GPUs.

7.5 Analysis Questions

There are many analysis questions that can be asked here. Some that interest me are:

- At what context length does splitting the context into chunks pay off?

- How should the number of chunks be chosen? Does dynamic really benefit workloads?
- What is the main limiting factor of each version (bandwidth, not enough blocks to keep the SMs busy, the combine overhead, etc.)?

8 Platform Choice

Since this is a memory bound workload and there are parallel workloads that I will exploit, the GPU is the right choice for the hardware. With a GPU, CUDA gives direct control over thread blocks and shared memory and thus I will be writing these kernels in CUDA C++. PyTorch will provide the reference, the SDPA baseline and TinyLlama integration. Also, I plan to use profiling tools like Nsight to help me analyze performance and optimize my implementations.

9 Schedule

Dates	**Tasks**
Nov 10–15	Set up GHC and local environments (CUDA, PyTorch extension build, TinyLlama via Hugging Face). Write PyTorch reference decode attention and benchmark. Implement and verify naive kernel (one block per query head).
Nov 16–22	Implement GQA grouped kernel and fixed-split kernel with combine step. Verify against reference. Run first sweeps over context length and profile.
Mon Nov 23	**Milestone report due** (60% goal complete).
Nov 23–29	Implement automatic chunk selection based on context length. Compare combine strategies and begin batched decode with variable-length requests.
Nov 30–Dec 6	Finish variable length batching and load balancing. Implement paged KV cache and if there is time, prepare for PSC.
Dec 7–10	Final benchmarks and final report.
Fri Dec 11	**Poster session**

Table 1: Project schedule

Disclosure of AI Tool Usage

- **Conceptual explanations** I asked questions to learn background material that was new to me and how they relate to my project idea. For example, the LLM helped me with understanding how TinyLlama's model architecture, a high level overview of how it's attention mechanism works, and the compute limitations of the 2080.
- **Project Scope** The AI helped me narrow down my project scope to a niche that would maximize overlap with course material.
- **Formatting, Equations and notation** The AI provided the standard attention equations and the list of notation definitions in LaTeX, which I included in the Background section. The AI also helped me with LaTex formatting (schedule table, display equations and hyperlinks).
- The syllabus was provided to the LLM so that it was aware of the academic integrity policy and did not violate it.